

AMSS 2026 — Lecture 1: Intro + the AI-mediated SDLC

Traian-Florin Șerbănuță

2026

Welcome

- ▶ AMSS 2026 — Analiza și Modelarea Sistemelor Software
- ▶ *Ediția AI-mediated*
- ▶ Instructor: Traian-Florin Șerbănuță
- ▶ Email: traian.serbanuta@unibuc.ro
- ▶ Semester: Fall 2026

Today's Agenda

1. Frame: the architect-critic loop + 5 course commitments
2. Live demo: an AI-driven design loop on a tiny scenario
3. What AI changes at each software development lifecycle (SDLC) stage
4. Tooling preview
5. Course logistics

AI Is Changing the SDLC

- ▶ Requirements, models, code, tests — all increasingly AI-generated.
- ▶ The skill is no longer *producing* artifacts. It's *directing* AI to produce them and *judging* what comes back.
- ▶ This course teaches both halves.

Your Turn: Have You Already Done This?

Hands up: who has used AI to generate code, requirements, or tests for a course?

Turn to your neighbour for 30 seconds: did you trust the output?
How did you check it?

The Architect-Critic Loop

The Architect-Critic Loop

- ▶ **Architect** — drive AI through the SDLC. Prompt, redirect, choose what to keep.
- ▶ **Critic** — read AI's output. Spot fabrication. Spot wrong multiplicity. Spot decorative pattern application.

Your Role: Both Halves

You are simultaneously:

- ▶ **Architect / director:** drive the work
- ▶ **Critic / reviewer:** read the work

The architect-half makes you the owner of the design. The critic-half makes you the owner of the quality.

Five Course Commitments (1/5)

AI is the default tool, not an exception.

Every artifact in this course — requirements, models, code, tests — is normally generated *with* AI. Your value-add is direction, judgment, and correction.

Five Course Commitments (2/5)

The trail is graded, not the running system.

What gets graded: directed-design narrative + final reviewed artifact set + oral defense. What does *not* get graded: running code, raw test pass-rates.

Five Course Commitments (3/5)

Implementation is a forcing function, not a deliverable.

Test-driven development (TDD) with AI is *required* for the project — but its output is *never* a grading line item.

The reflection on what TDD revealed about your spec is what gets read.

Five Course Commitments (4/5)

Defensible literacy floor — Critique, Rationale, Traceability.

In the oral defense, *unaided*, you must demonstrate:

- ▶ **Critique** — Read & critique any AI-generated UML diagram on the spot.
- ▶ **Rationale** — Articulate why you directed AI a certain way and what you accepted or rejected.
- ▶ **Traceability** — Defend traceability across your project: requirement → use case → class → state/sequence → test.

Five Course Commitments (5/5)

Tooling parity.

Every student uses the same canonical agentic tooling: one editor extension, one model endpoint, one config file.

You may *also* use Claude Code, Copilot, Cursor on personal accounts — but graded artifacts must reproduce on the canonical setup.

Watch For Both Moves

In the demo coming up:

- ▶ When I'm typing prompts → that's the **architect** move.
- ▶ When I'm reading AI's output and pointing at problems → that's the **critic** move.

You're going to do both halves yourself, starting in Lab 1.

Demo: Library Kiosk

Live AI design loop. Watch the architect move and the critic move.

Prompt to AI: *“Generate a UML class diagram for a small library kiosk: users borrow and return books; staff register returns; books can be reserved while on loan.”*

What AI Changes at Each SDLC Stage

The next 7 slides walk the SDLC stage by stage.

For each: **what AI does well** vs. **where AI fails**.

This is also the rest of your semester — each stage previews a future week.

Requirements Gathering (Week 2 preview)

AI does well:

- ▶ Drafts long lists of plausible-looking requirements fast.
- ▶ Surfaces stakeholder roles and use cases you might not have considered.

AI fails at:

- ▶ Fabricating non-functional requirements without grounding.
- ▶ Vague non-functional requirements (NFRs) (“the system shall be performant”).
- ▶ Over-specifying — generating 200 requirements where 20 would do.

Test/Spec Design (Week 3 preview)

AI does well:

- ▶ Generates test scaffolds from a clear specification.
- ▶ Surfaces edge cases the spec doesn't mention.

AI fails at:

- ▶ Producing tests for vague specs — they pass for the wrong reasons.
- ▶ Over-fitting tests to a particular implementation.
If AI can't produce a passing test from your spec, your spec is too vague.

Structural Modeling (Weeks 4-5 preview)

AI does well:

- ▶ Drafts a class diagram from a one-page spec in seconds.
- ▶ Suggests reasonable class names and relationships.

AI fails at:

- ▶ Wrong multiplicities (you saw one in the demo).
- ▶ Fabricated classes with no behavior.
- ▶ Conflating abstract concepts with concrete instances.

Behavioral Modeling (Weeks 6-7 preview)

AI does well:

- ▶ Generates sequence diagrams for happy-path scenarios.
- ▶ Sketches state machines for object lifecycles.

AI fails at:

- ▶ Fabricating messages between objects that don't exist.
- ▶ Missing guards on state transitions.
- ▶ Orphan states / unreachable transitions.

Patterns (Weeks 8-9 preview)

AI does well:

- ▶ Recognizes textbook pattern situations and applies them.
- ▶ Knows the Gang of Four (GoF) vocabulary.

AI fails at:

- ▶ Decorating code with patterns that solve nothing.
- ▶ Overusing patterns where simple code would do.
- ▶ Calling something “Visitor” when it’s just an `if/else` on a type tag.

Traceability, Quality & Evaluation (Weeks 10-11 preview)

AI does well:

- ▶ Cross-references artifacts on demand.
- ▶ Spots structural gaps if asked the right question.

AI fails at:

- ▶ Maintaining traceability across iterations.
- ▶ Self-evaluating its own output.
- ▶ Catching consistency issues without being prompted.

Coding (Out of Scope Here)

Implementation is a *parallel course's* domain — not ours.

We use it as our **forcing function**: TDD-with-AI is required for the project, but its output is never graded.

What gets read is your *reflection on what the loop revealed*.

Back to Critique, Rationale, Traceability

Every stage above gets evaluated through the same three abilities:

- ▶ **Critique** — read & critique what AI produced.
- ▶ **Rationale** — articulate why you directed AI the way you did.
- ▶ **Traceability** — defend the trace from one stage to the next.

This is the literacy floor. The oral defense tests it directly.

What You Just Saw: The Course Tooling

- ▶ **Editor:** VS Code with the Continue.dev extension.
- ▶ **Endpoint:** the canonical course model endpoint (configured in `tooling/.continue/config.yaml`).
- ▶ **Mode:** agentic chat — file-and-repo aware, not browser ChatGPT.

This is what every student in the cohort runs.

Where We Install: Lab 1

- ▶ Lab 1 (Week 2): hands-on tooling onboarding. Don't try alone before then.
- ▶ Setup guide: `tooling/SETUP.md` in the course repo.
- ▶ Estimated time: ~30 min on a working laptop.

Tooling Parity (BYO Allowed, On Top)

You may also use, on your personal accounts:

- ▶ Claude Code, GitHub Copilot, Cursor — anything you have access to.

But: every graded artifact must reproduce on the canonical setup.

This is what lets the examiner reproduce your work during the oral defense.

Schedule

- ▶ **14 teaching weeks:** 12 lecture weeks (~100 min each), plus a project workshop (Week 13) and final presentations (Week 14).
- ▶ **7 labs**, biweekly (Weeks 2, 4, 6, 8, 10, 12, 14 — 100 min each).
- ▶ Course landing page:
`traiansf.github.io/class/amss2026/`.

Final Grade

Component	Points
Project	8
Attendance	1
<i>Din oficiu</i>	1
Total	10

The Project

- ▶ **Teams of 3-5 students.**
- ▶ Teams not formed by 1 November are randomized.
- ▶ Domain announced by **31 October 2026**.
- ▶ Multiple teams may share a domain — the directed-design trails differ.
- ▶ Each student owns a **slice** of their team's system.
- ▶ December checkpoint (Lab 5): 1 pt. Remaining 7 pts across the oral defense, directed-design narrative + defect log, documentation, and design patterns — see the project rubric.

Oral Defense — Critique, Rationale, Traceability

Cold defense, 3 of 8 project points. *Unaided*, you must:

- ▶ **Critique** — Read & critique any AI-generated UML diagram from any team's repo.
- ▶ **Rationale** — Articulate rationale for your own design decisions.
- ▶ **Traceability** — Defend traceability across your project's slices.

This is the integrity check.

Resit Exam (Restanță)

For students who fail the regular evaluation path:

- ▶ Single 90-min written paper.
- ▶ Format: critique an AI-generated artifact set + short-answer rationale + traceability walk.
- ▶ **No AI in the room.**
- ▶ Tests the same competence (Critique, Rationale, Traceability) as the oral defense.

Academic Integrity

- ▶ AI use is **expected** for almost every artifact in this course.
- ▶ The *directed-design narrative* is your record of what you did with AI.
- ▶ The *cold defense* is the integrity check — examiners verify you understand your own work.
- ▶ One-page statement attached to the syllabus (circulated before Week 2).

Communication

- ▶ *Teams link / mailing list — to be circulated before Week 2.*
- ▶ Office hours: by appointment (email).
- ▶ Course repo: GitHub (link circulated with the team-formation announcement).

That's It For Today

- ▶ Next week: requirements with AI.
- ▶ Lab 1 (Week 2 lab slot): tooling onboarding + first AI-driven requirements drill.

Questions?